CSS Advanced Selectors

Pseudo-Elements Guide

Create virtual elements and style specific parts of content

Easy English • Practical Examples • Ready to Use

Contents

Chapter 1

Introduction to Pseudo-Elements

1.1 What are Pseudo-Elements?

Pseudo-elements are CSS selectors that create virtual elements or allow you to style specific parts of an element. Unlike pseudo-classes which target based on state or position, pseudo-elements create new content or target specific portions of existing content. Pseudo-elements use double colons (`::`) in modern CSS.

Think of it as: "Create virtual content or style specific parts of an element without adding HTML."

Key Concept

Pseudo-elements let you add decorative content with CSS, style the first letter or line of text, create quote marks, and more—all without adding extra HTML elements.

1.2 Pseudo-Classes vs Pseudo-Elements

- **Pseudo-Classes (:):** Style elements based on state or position (e.g., :hover, :first-child)
- **Pseudo-Elements (::):** Create virtual elements or style specific text portions (e.g., ::before, ::after)

1.3 Why Use Pseudo-Elements?

- Add decorative content without cluttering HTML
- Style specific parts of text independently
- Create before/after effects and overlays
- Reduce the need for extra HTML elements
- Build complex designs with minimal markup
- Improve website performance

1.4 Browser Support

Modern browsers support the `::` syntax. The old single colon (`:`) syntax still works for backward compatibility but is considered outdated.

Chapter 2

Content Pseudo-Elements

2.1 1. ::before

2.1.1 Explanation

The `::before` pseudo-element creates a virtual element as the first child of the targeted element. You can use the `content` property to add text, images, or decorative symbols.

In simple words: "Add content before an element"

2.1.2 Example

CSS Code

```
p::before {
    content: " ";
    color: green;
    font-weight: bold;
}
```

HTML:

```
<p>This is a paragraph</p>
```

Result: A green checkmark appears before the paragraph text.

More complex example with styling:

CSS Code - Button with Icon

```
button::before {
    content: " ";
    margin-right: 8px;
}

button {
    padding: 10px 20px;
    background-color: blue;
    color: white;
}
```

HTML:

```
<button>Search</button>
```

Result: A search icon appears before the ”Search” text inside the button.

2.2 2. ::after

2.2.1 Explanation

The `::after` pseudo-element creates a virtual element as the last child of the targeted element. Like `::before`, you use the `content` property to add content.

In simple words: "Add content after an element"

2.2.2 Example

CSS Code

```
a[target="_blank"]::after {
    content: " \2197";
    font-size: 0.8em;
    margin-left: 4px;
}
```

HTML:

```
<a href="https://example.com" target="_blank">
    External Link
</a>
```

Result: A small arrow symbol appears after external links to indicate they open in a new tab.

Another example - adding required field indicators:

CSS Code - Form Labels

```
label::after {
    content: " *";
    color: red;
}
```

HTML:

```
<label>Email Address</label>
```

Result: A red asterisk appears after the label text.

2.3 3. Content Property Values

The `content` property accepts various values:

2.3.1 Text Content

CSS Code

```
p::before {
    content: "Note: ";
}
```

2.3.2 Unicode Characters

]

```
li::before {
    content: "\2714"; /* Checkmark */
}
```

]

```
p::before {
    content: " ";
}
```

]

```
a::before {
    content: url("icon.png");
    margin-right: 5px;
}
```

]

```
a::after {
    content: " (" attr(href) ")";
    font-size: 0.8em;
    color: gray;
}
```

<

HTML:
b href=”/page”>Link
Result: Displays as: ”Link (/page)”

Chapter 3

Text Pseudo-Elements

3.1 1. ::first-letter

3.1.1 Explanation

The `::first-letter` pseudo-element applies styles to the first letter of text inside an element.

In simple words: "Style the first letter of text"

3.1.2 Example

CSS Code - Drop Cap Style

```
p::first-letter {
    font-size: 2em;
    font-weight: bold;
    color: blue;
    float: left;
    margin-right: 8px;
}
```

HTML:

```
<p>Lorem ipsum dolor sit amet...</p>
```

Result: The first letter "L" becomes large, bold, and blue, creating a drop cap effect.

`::first-letter` works with: font, color, background, text-decoration, text-transform, letter-spacing, word-spacing, line-height, float, margin, padding, border

3.2 2. ::first-line

3.2.1 Explanation

The `::first-line` pseudo-element applies styles to the first line of text inside an element.

In simple words: "Style the first line of text"

3.2.2 Example

<

HTML:
b> This is the first line of the paragraph. This is the second line. </p>

Result: Only the text in the first line becomes bold, dark blue, and uppercase. The styling adjusts dynamically as the window resizes.

`::first-line` is responsive. The first line changes based on the viewport width. Only certain CSS properties work with it.

3.3 3. ::selection

3.3.1 Explanation

The `::selection` pseudo-element applies styles to text that is selected (highlighted) by the user.

In simple words: "Style highlighted text when user selects it"

3.3.2 Example

<
HTML:
b>Select this text to see the custom highlighting</p> Also try selecting this link

Result: When you highlight text with your mouse, it gets custom colors instead of the browser default.

`::selection` has excellent browser support. Use single colon (`:selection`) for older browsers.

Chapter 4

Advanced Pseudo-Elements

4.1 1. ::placeholder

4.1.1 Explanation

The `::placeholder` pseudo-element styles the placeholder text inside form inputs.

In simple words: "Style the gray placeholder text in input fields"

4.1.2 Example

<
HTML:
bnput type="text" placeholder="Enter your name"> <textarea placeholder="Enter your message"></textarea>

Result: The placeholder text appears in a custom color and style.

4.2 2. ::marker

4.2.1 Explanation

The `::marker` pseudo-element styles the marker (bullet or number) of list items.

In simple words: "Style the bullets or numbers in lists"

4.2.2 Example

]

```
ul li::marker {
    color: blue;
    font-weight: bold;
}

ol li::marker {
    color: red;
    font-size: 1.2em;
}
```

<
HTML:
bl> Item 1 Item 2
 First Second

Result: Bullets in unordered lists become blue and bold. Numbers in ordered lists become red and larger.

4.3 3. ::cue

4.3.1 Explanation

The `::cue` pseudo-element styles text cues in video captions.

In simple words: "Style video captions and subtitles"

4.3.2 Example

]

```
video::cue {
    background-color: rgba(0, 0, 0, 0.8);
    color: white;
    font-size: 16px;
}
```

Note: `::cue` is used with video elements that have captions.

Chapter 5

Practical Examples

5.1 Example 1: Blockquote with Decorative Quotes

Result: A professional blockquote with large decorative quote marks at the beginning and end.

5.2 Example 2: Icon Links

Result: Each link type displays with an appropriate icon.

5.3 Example 3: Drop Cap Typography

Result: The first letter of the article is enlarged and floated, creating a classic drop cap effect.

5.4 Example 4: Warning Messages with Icons

5.5 Example 5: Code Block with Line Numbers

5.6 Example 6: Custom Form Styling

/

Chapter 6

Quick Reference

6.1 All Pseudo-Elements Summary

Pseudo-Element	What It Does	Example
::before	Add content before	`p::before`
::after	Add content after	`p::after`
::first-letter	Style first letter	`p::first-letter`
::first-line	Style first line	`p::first-line`
::selection	Style selected text	`::selection`
::placeholder	Style placeholder	`input::placeholder`
::marker	Style list markers	`li::marker`
::cue	Style video captions	`video::cue`

6.2 Content Property Values

Value Type	Example
Text	`content: "Text";`
Emoji	`content: " ";`
Unicode	`content: "713";`
Image	`content: url("icon.png");`
Attribute	`content: attr(href);`

Chapter 7

Tips and Best Practices

7.1 Tip 1: Use ::before and ::after for Decoration

Use pseudo-elements for purely decorative content, not essential information:
b Good - Decorative */ button::after content: ” →”;
/* Bad - Essential content */ /* Don’t put important info in ::before/::after */

7.2 Tip 2: Combine with Content Property Wisely

Remember the `content` property is required for pseudo-elements to display:

```
/* WRONG - No content, nothing displays */
p::before {
    color: red;
}

/* RIGHT - Must have content property */
p::before {
    content: "";
    color: red;
}
```

7.3 Tip 3: Use attr() to Display HTML Attributes

Display attribute values dynamically:

```
/* Show the href in a link's title */
a::after {
    content: " (" attr(href) ")";
    font-size: 0.8em;
}

/* Show data attributes */
div::after {
    content: attr(data-tooltip);
}
```

7.4 Tip 4: Accessibility Considerations

Screen readers may read content from pseudo-elements:

```
/* Good - Decorative only */
button::after {
    content: " ";
    width: 10px;
    height: 10px;
}

/* Provide alternative text if important */
a[rel="external"]::after {
    content: " (external link)";
}
```

7.5 Tip 5: Use ::selection for Brand Consistency

Customize text selection to match your brand:

```
::selection {
    background-color: #007bff;
    color: white;
}

/* Different for links */
a::selection {
    background-color: #0056b3;
}
```

7.6 Tip 6: ::first-letter and ::first-line Compatibility

Not all CSS properties work with text pseudo-elements:

```
/* Works with ::first-letter */
p::first-letter {
    font-size: 2em;
    color: blue;
    float: left;
}

/* Limited with ::first-line */
p::first-line {
    font-weight: bold;
    color: blue;
}
```

Chapter 8

Common Mistakes to Avoid

8.1 Mistake 1: Using Single Colon with Modern Elements

```
/* OLD - Still works but outdated */
p:before { content: ""; }

/* NEW - Correct syntax */
p::before { content: ""; }
```

8.2 Mistake 2: Forgetting the content Property

```
/* WRONG - Nothing displays */
p::before {
    color: red;
    font-size: 1.5em;
}

/* RIGHT - Must include content */
p::before {
    content: →"";
    color: red;
    font-size: 1.5em;
}
```

8.3 Mistake 3: Using Pseudo-Elements for Important Content

```
/* WRONG - Inaccessible to screen readers */
label::after {
    content: "(required)";
}

/* BETTER - Use HTML for content */
<label>Email <span>(required)</span></label>

/* Or use aria attributes */
```

```
<input aria-required="true">
```

8.4 Mistake 4: Complex Layouts with ::before/::after

```
/* Avoid using pseudo-elements for layout */
/* Use them for decoration only */

/* OKAY - Decorative */
.box::before { content: →""; }

/* WRONG - For layout, add HTML elements */
```

8.5 Mistake 5: Ignoring Browser Support

```
/* ::marker has lower support */
li::marker {
    color: blue;
}

/* Check Can I Use for compatibility */
/* Use fallbacks when needed */
```

Chapter 9

Browser Support

9.1 Pseudo-Element Support Table

Pseudo-Element	Chrome	Firefox	Safari
::before	All	All	All
::after	All	All	All
::first-letter	All	All	All
::first-line	All	All	All
::selection	All	All	All
::placeholder	57+	51+	10.1+
::marker	86+	68+	Limited
::cue	26+	31+	Limited

Chapter 10

Summary

What You Learned

8 Main Pseudo-Elements:

1. **::before** - Add content before element
2. **::after** - Add content after element
3. **::first-letter** - Style first letter
4. **::first-line** - Style first line
5. **::selection** - Style highlighted text
6. **::placeholder** - Style form placeholders
7. **::marker** - Style list markers
8. **::cue** - Style video captions

Key takeaways:

- Pseudo-elements create virtual content or style specific text parts
- Use `::before` and `::after` for decorative content only
- The `content` property is required for content pseudo-elements
- Text pseudo-elements (::first-letter, ::first-line) work with specific properties
- Use `attr()` to display HTML attributes
- Pseudo-elements improve design while keeping HTML clean
- Always consider accessibility when using pseudo-elements

Master pseudo-elements to create sophisticated designs with minimal HTML!

Chapter 11

Practice Exercises

11.1 Exercise 1: Decorative List

Create an unordered list where:

- Each item has a custom bullet before it
- Add a text label after the item

11.2 Exercise 2: Article with Drop Cap

Build an article where:

- First letter is enlarged and floated (drop cap)
- First line is in bold
- Text selection has custom styling

11.3 Exercise 3: Icon Links

Create links that:

- Show icons before each link using ::before
- Different icons for external links, emails, and PDFs
- Add arrow after external links

11.4 Exercise 4: Custom Form Styling

Design a form where:

- Labels have red asterisks for required fields
- Placeholder text has custom styling
- Success/error messages have icons before them

11.5 Exercise 5: Blockquote with Quotes

Create a styled blockquote with:

- Large decorative opening quote before
- Large decorative closing quote after
- Special styling for the first line
- Custom selection color

Complete these exercises to master pseudo-elements!